

软件工程基础第 1 章——软件工程概述学习笔记

基于 224 页课件精炼整理 | 2026 年 6 月

1 系统工程概述

1.1 系统的概念

[p.4-5]

- **系统定义**：由两个或以上有机联系、相互作用的要素所组成，具有特定功能、结构和环境的整体。
- **系统三要素**：
 - **功能 (Function)**：系统存在的作用和价值（相互作用）
 - **结构 (Structure)**：组成要素之间相对稳定的联系方式
 - **环境 (Environment)**：系统边界之外，与系统相互影响的部分（依存关系）
- **现实系统的复杂性来源**：要素多维、动态变化、人机交互、开放适应、学习重组。 [p.6]

1.2 系统工程 (Systems Engineering)

[p.8-9]

- **定义**：从整体出发，合理开发、管理、运行和改进一个大规模复杂系统所需的思想、理论、方法和技术的总称。
- **核心活动**：需求分析 → 系统设计 → 子系统开发 → 系统集成 → 系统测试 → 系统部署。
- **三大过程**：采购（决定范围、预算、进度、高层需求）、开发、运行。
- **理论基础**：运筹学、系统理论、管理科学。

1.3 典型案例：都江堰水利工程

[p.11-17]

公元前 256 年，李冰率众修建。以系统工程思想实现防洪、灌溉、清淤三项需求。

系统工程活动	都江堰体现	技术手段
采购（需求分析）	实地考察，确定高层需求	与自然和谐共存
开发（系统设计）	鱼嘴分水、飞沙堰排沙、宝瓶口引水	火烧水浇开凿（热胀冷缩）
运行（维护管理）	岁修制度（淘滩作堰）	就地取材，造价低廉

- **总体目标最优化**：分四六、平潦旱。枯水期内江六成外江四成灌溉；丰水期反之避免洪灾。 [p.15-16]

1.4 科学、技术与工程的关系

[p.18]

- **科学 (Science)**：以发现为核心，探索自然本质和运行规律，出于好奇心。
- **技术 (Technology)**：以发明为核心，改善人类生活的手段（方法、装置、工具、过程）。

- **工程 (Engineering)**: 集成科学和技术解决实际问题, 在边界条件下进行集成优化和综合优化 (人、财、物)。

2 软件特性与分类

2.1 软件的本质特征

[p.23]

- 软件是计算机系统中看不见、摸不着的逻辑部分, 以**程序、数据和文档**的形式出现。
- **四大特性**:
 1. 软件不是”制造”产生的, 而是”研发”出来的 → 项目管理和产品保护困难
 2. 软件不会被”用坏” → 维护困难 (不能通过重复制造解决)
 3. 软件大多是”定制”的 → 质量和效率受影响
 4. 软件成本难于估计 → 项目计划易失效

2.2 软件分类

[p.25-26]

分类维度	类别	说明与示例
按功能划分	系统软件	与硬件紧密交互, 如操作系统、设备驱动
	支撑软件	协助开发工具, 如编译器、测试工具
	应用软件	面向最终用户功能, 如播放器、微波炉控制
按服务对象	项目软件	受特定用户委托开发 (如空中交通管制系统)
	产品软件	直接面向市场开发 (如游戏软件)

3 软件危机与软件工程

3.1 典型案例: IBM OS/360

[p.28]

- 1961 年 IBM 启动 360 系统计划, 投资约 50 亿美元。
- 软件工程师超 2000 人, 花费超 5 亿美元 (超过硬件研发费用)。
- 360 操作系统用 5000 人年, 最终未完全实现设想。
- 负责人 Frederick Brooks 据此撰写《人月神话》(*The Mythical Man-Month*)。

3.2 软件危机 (Software Crisis)

[p.29]

20 世纪 60-70 年代, 软件作坊模式暴露严重问题:

- **质量差**: 可靠性难以保证 (bug 多)

- **成本失控**：很少在预算内完成
- **进度拖延**：开发周期拖得很长
- **维护困难**：维护人员和费用不断增加
- **内因**：软件本身的特点（逻辑性、复杂性）
- **外因**：开发维护及组织管理不当

3.3 软件工程（Software Engineering）

[p.31-34]

- **1968 年 NATO 会议**正式提出” 软件工程” 术语。
- **NATO 定义**：用来建立和使用合理的工程原则，以经济地获取可靠的、且在真实机器上可高效工作的软件。
- **计算机百科全书定义**：应用计算机科学、数学及管理科学等原理，以工程化方法制作软件的工程。
- **核心要素**：
 - 性质：指导软件开发和维护的工程性学科
 - 理论基础：计算机科学、管理科学和数学
 - 手段：工程化概念、原理、技术和方法
 - 目的：以较少代价获取高质量软件

3.4 软件工程发展阶段

[p.35-36]

阶段	时间	特点
程序设计阶段	20 世纪 50-60 年代	专家个体手工劳动；程序写法不受限制；软件受制于硬件
程序系统阶段	20 世纪 60-70 年代	软件作坊；注重程序设计风格；软件供不应求 → 软件危机
软件工程阶段	20 世纪 70 年代至今	工程化、多学科交叉
智能软件工程阶段	21 世纪 20 年代至今	自动化；AI 驱动开发

3.5 软件产业新增长点

[p.37-40]

- **云计算**（算力驱动）
- **物联网**（大数据驱动）
- **人工智能**（模型 + 数据 + 算力；生成式 AI）
- **前沿方向**：群智化软件工程、开发运维一体化（DevOps）、云计算软件工程、智能化软件工程、大数据驱动软件工程、区块链软件工程。

4 软件可信性（Dependability）

4.1 可信性概念与案例

[p.45–47]

可信性反映系统向其用户提供预期服务水平的能力。

典型案例	原因分析
阿里亚娜 V 型火箭爆炸（1996，5 亿欧元损失）	64 位浮点数转 16 位有符号整数溢出
爱国者导弹拦截失败（1991 年海湾战争）	时钟累积误差导致无法预测导弹弹道（十进制 1/10 用二进制表示产生微精度误差）
12306 订票系统崩溃	系统架构和客票发放机制缺陷，无法支持大并发量

4.2 软件故障类型

- [p.48]
- 退化故障（耗损故障）：性能随时间衰减引起——软件故障不是退化故障。
 - 设计故障：系统设计中存在的缺陷，在软件生命周期任何时刻都可能出现。
 - 拜占庭故障（Byzantine Fault）：行为完全无法预测的故障，由恶意攻击或软件错误导致。

4.3 可信性属性体系

[p.50–57]

属性	定义	关键公式/因素
可靠性 (Reliability)	给定条件下在 $[0, t]$ 内持续正确服务的概率 $R(t)$	$R(t)$ 为成功运行概率；失效概率 $Q(t) = 1 - R(t)$
可用性 (Availability)	t 时刻能正确运行的概率 $A(t)$	稳态可用度 $A_{\infty} = \frac{MTTF}{MTTF + MTTR}$
防危性 (Safety)	单位时间失效损失期望值 < 规定阈值	$Risk = \sum_i P_i(\text{失效}) \times L_i(\text{失效})$
完整性 (Integrity)	不存在未经授权的修改或操控	静态完整性 + 动态完整性；预防/检测/恢复
安全性 (Security)	CIA 三要素：机密性、完整性、可用性	安全性（事前遵守规则）vs. 防危性（事后补救）
可维护性 (Maintainability)	失效后给定时间内可修复的概率 $M(t)$	$M(t) = 1 - e^{-\mu t}$ ；影响因素：可理解性、易操作性

- Trust（信任）= 可信赖的各质量属性的系统化组合。[p.50]

4.4 可信性工程（Dependability Engineering）

- [p.58–64]
- 主要目标：正确处理软件系统中的故障，四种处理手段：
- 故障避免（Fault Avoidance）：引入前预防——严格遵循工程方法、原型技术、形式化方法、编码标准。[p.59]
 - 故障移除（Fault Removal）：引入后检测修复——人工审核、软件测试、模型检测、系统仿真。[p.60]
 - 容错（Fault Tolerance）：故障存在时保持可接受服务——使用冗余技术。[p.61]

- 时间冗余：重复计算/传输并对结果比较
- 空间冗余：增加多余部件/数据项 + 表决器

4. 故障预测 (Fault Prediction)：评估系统行为结构预测未来故障。[p.63]

- 定性预测 vs. 定量预测；静态预测 vs. 动态预测

4.5 拜占庭容错

[p.62]

- 核心思想：对于每一个收到命令的将军，要去询问其他人收到的命令是什么。
- 通过多数表决机制，在存在叛徒（拜占庭故障）的情况下达成一致。

5 软件工程基本体系

5.1 四个研究内容

[p.69-71]

- 标准和规范 (Standards & Norms)：使各项工作有章可循，提高效率和质量。
- 过程与模型 (Process & Model)：过程是活动和任务，模型将过程有机结合。
- 方法和技术 (Methods & Techniques)：方法是看待系统的立场，技术是方法的具体实现。
- 工具和环境 (Tools & Environment)：为开发维护提供自动/半自动支持；环境是工具的有机集成。

层次结构（以软件质量为核心）：标准和规范 → 过程与模型 → 方法和技术 → 工具和环境 → 软件质量。

5.2 软件工程七条基本原理

[p.73]

1. 用分阶段的生存周期计划严格管理
2. 坚持进行阶段评审
3. 实施严格的产品控制
4. 采用现代程序设计技术
5. 结果应能清楚地审查
6. 开发小组的成员应该少而精
7. 不断改进软件工程实践的必要性

6 软件开发方法

6.1 方法总览

[p.78-79]

软件开发方法 = 开发步骤（含技术和符号）+ 软件文档格式 + 开发方案评价标准。

程序 = 数据结构 + 算法 (Niklaus Wirth)，不同方法从不同视角看待程序：

方法	视角	核心思想
结构化方法	以功能为中心（算法角度）	功能分解、自顶向下、逐步求精
面向对象方法	以数据为中心（数据结构角度）	对象 + 类 + 继承 + 消息通信
面向方面方法	横切关注点分离	将散布的关注点模块化为“方面”
形式化方法	数学分析推理	精确的数学描述和验证

6.2 结构化方法 (Structured Method)

[p.81-85]

- **核心思想：**自顶向下、逐步求精、单入口单出口，利用抽象和功能分解处理复杂性。
- **发展脉络：**结构化程序设计 → 结构化设计 → 结构化分析。
- **代表方法：**面向数据流图方法、IDEF0、Jackson 方法、LCP 方法。
- **结构化程序理论** (Böhm-Jacopini, 1966)：任何逻辑问题均可仅用顺序、选择和循环结构解决。
 - 中国唐稚松院士 1965 年在《数学学报》发表类似结论，早于 Böhm-Jacopini 一年。
- **Dijkstra** (结构程序设计之父)：1968 年发表“Goto Statement Considered Harmful”，提出 goto 有害论、信号量和 PV 原语、最短路径算法、银行家算法。
- **优点：**方法简单实用、技术成熟、应用广泛。
- **缺点：**不适应大规模/复杂项目；难于解决重用；难以适应需求变化；维护复杂。

6.3 面向对象方法 (Object-Oriented Method)

[p.86-89]

- **核心思想：**尽可能模拟人类习惯的思维方式——对象、类、继承、消息通信。
- **发展脉络：**OOD → OOP → OOA。
- **起源：**
 - Simula67 (Dahl & Nygaard, 2001 图灵奖)：最早引入对象和类概念
 - Smalltalk (Alan Kay, 2003 图灵奖)：最纯的 OOP 语言，引入对象、方法、实例、动态联编、单继承
- **优点：**对象比功能更易理解；系统更稳定；易处理需求模糊/变化；便于重用；封装便于修改维护。
- **不足** (Joe Armstrong)：类作为复用单元有时太小；继承增加类间耦合；理论不如结构化成熟；比结构化方法复杂。
- **核心优势：**问题空间和软件空间一致，系统成分直接映射问题域。[p.89]

6.4 面向方面方法 (Aspect-Oriented Method)

[p.91-95]

- **横切关注点** (Cross-cutting Concerns)：散布在功能模块或对象类中的横切行为。
 - 结构化方法：改变数据需修改许多功能模块
 - 面向对象方法：改变某一功能需修改许多对象
 - 面向方面方法：利用关注点分离，每个成分做且只做一件事

- 方面（Aspect）= 横切关注点的模块化实现。
- 切入点（Pointcut）：连接点（Join Point）的集合，横切行为的插入点。
- 通知（Advice）：在切入点处增加的行为。类间声明（Inter-type Declaration）：在已有类型中添加新方法/属性/接口。[p.93]
- 五种关注点（Ian Sommerville）：功能性、服务质量、政策、系统、机构。[p.94]

6.5 形式化方法（Formal Method）

[p.97–99]

- 核心思想：通过严格的、规范化的数学理论经分析、推理和转换得到系统。
- 代表方法：VDM、RAISE、Event-B、Z、B 方法。
- 两大研究内容：
 - 形式规约（Formal Specification）：对程序”做什么”的精确数学描述
 - 形式验证（Formal Verification）：正确性证明、模型检测、模拟
- 优点：完整、一致、无二义性；支持形式推理；便于软件自动生成。
- 缺点：一般人不易接受；灵活性差；难以与工程化过程结合；支持工具少。

6.6 集成方法：SOFL

[p.101–103]

- **SOFL**（Structured Object-based Formal Language）：刘少英（日本法政大学）提出。
- 结构化方法做需求分析规约 + 面向对象方法做设计实现 + 形式化方法保证高质量规约和验证。
- **Agile-SOFL**：敏捷形式化工程方法，兼顾生产率和可靠性。[p.103]

7 软件工具与环境

7.1 软件工具分类

[p.108–111]

工具类别	示例
软件开发工具	需求分析工具、软件设计工具、软件编码工具、软件测试工具、文档写作工具
软件维护工具	版本控制工具、文档分析工具、逆向工程工具、再工程工具
项目管理工具	项目管理工具、配置管理工具、软件评价工具、开发信息库工具

- 问题：单一工具只支持一种活动；工具界面不统一，切换需人工操作。[p.112]

7.2 集成开发环境（IDE）

[p.114–116]

- 构成：软件工具集 + 环境集成机制。

- **IPSE** (Integrated Project Support Environment): 三层架构——核心层 (宿主机硬件/OS)、基本层 (各种开发工具)、应用层 (各种方法开发工具)。
- **ECMA 参考模型**: 数据存储服务、数据集成服务、任务管理服务、消息服务、用户界面服务、工具槽。
- 代表性系统: 北京大学青鸟系统 (Jade Bird System)。

8 AI 辅助软件开发

8.1 AI 模型基础

[p.119]

- **生成对抗网络 (GAN)**: 生成器和判别器对抗训练
- **Transformer 模型**: 注意力机制 (Attention Mechanism)
- **扩散模型 (Diffusion Model)**: 正向加噪 → 逆向除噪

8.2 主要 AI 编码工具

[p.121–122]

- **MarsCode** (字节跳动): marscode.cn
- **通义灵码** (阿里): lingma.aliyun.com
- **DeepSeek**: chat.deepseek.com
- **腾讯 Copilot**: copilot.tencent.com
- **小浣熊代码**: xiaohuanxiong.com/code

8.3 百度千帆大模型 API 调用

[p.123–125]

- 注册百度千帆 ModelBuilder 平台, 创建 API Key。
- 使用 OpenAI SDK 兼容接口调用 DeepSeek-V3/R1 等模型。
- 支持的模型列表: cloud.baidu.com/doc/WENXINWORKSHOP/s/Fm2vrveyu

8.4 VSCode + Cline + DeepSeek 开发环境

[p.126]

AI 辅助编码环境搭建方案: VSCode 编辑器 + Cline 插件 + DeepSeek API 作为 AI 提供者。

9 Flask Web 框架实例教学

9.1 Flask 框架概述

[p.128]

- **特点**: 微型框架 (Micro-framework)、可扩展、MVC 架构、Python 编写。

- 两大核心依赖：Werkzeug（路由等）、Jinja2（模板引擎）。

9.2 环境搭建

[p.130]

- 安装 virtualenv 创建虚拟环境：`pip install virtualenv`
- 构建虚拟环境：`virtualenv venv`，激活后安装 Flask：`pip install Flask`
- 依赖管理：`pip freeze > requirements.txt`（导出）/`pip install -r requirements.txt`（恢复）

9.3 基本应用结构

[p.131–133]

- WSGI 应用：Flask 实例化 + `@app.route()` 路由装饰器 + 视图函数 + `app.run()`。
- 项目目录结构：app 包（templates 模板、static 静态文件、main 程序包）、migrations 数据迁移、tests 测试、venv 虚拟环境。

9.4 路由与视图函数

[p.138–141]

- 路由（Route）：URL 和视图函数之间的映射关系。
- 核心组件：`url_map`（Map 类，URL → endpoint 映射）和 `view_functions`（endpoint → 视图函数映射）。
- 动态路由：`@app.route('/<int:id>/')`，支持 string、int、float 等类型参数。
- HTTP 方法：GET（获取资源）、POST（提交数据）、PUT（上传内容）、DELETE（删除资源）。

9.5 模板引擎（Jinja2）

[p.145–154]

- 目的：分离业务逻辑和展示逻辑，提高可维护性。
- 变量渲染：`{{ variable }}`，支持过滤器（如 `{{ name|upper }}`）。
- 控制结构：`{% if %} ... {% endif %}`、`{% for %} ... {% endfor %}`。
- 宏（Macro）：类似 Python 函数，可独立存储于文件，通过 `{% import %}` 导入。
- 模板继承：`{% extends "base.html" %} + {% block %} ... {% endblock %} + {{ super() }}`。

9.6 Bootstrap 集成

[p.155–158]

- 安装：`pip install flask-bootstrap`
- 初始化：`Bootstrap(app)`
- 模板继承"`bootstrap/base.html`"，提供 navbar、content 等 block。

9.7 Web 表单 (Flask-WTF)

[p.161–167]

- 安装: `pip install flask-wtf`
- 创建表单类: 从 `FlaskForm` 继承, 定义字段属性。
- CSRF 保护: 配置 `CSRF_ENABLED = True` 和 `SECRET_KEY`。
- 表单验证: `form.validate_on_submit()`, 通过 `request.form` 访问表单数据。

9.8 数据库 (Flask-SQLAlchemy)

[p.168–185]

- ORM 框架: 将类和对象转换成 SQL, 使用数据 API 执行 SQL 并获取结果。
- 配置: `SQLALCHEMY_DATABASE_URI` 指定数据库 URL。
- 模型定义: 继承 `db.Model`, 定义 `db.Column` 字段、`db.relationship` 关系、`db.ForeignKey` 外键。
- 查询操作: `Model.query.all()`、`filter()`、`filter_by()`、模糊查询 `like()`、逻辑查询 `and_()`/`or_()`/`not_()`。
- MTV 架构: 浏览器 → URL 调度 → 视图 (View) → 模型/ORM (Model) → 数据库。[p.180]

9.9 Flask Shell 与 ORM 操作

[p.184–185]

- 设置环境变量 `FLASK_ENV=development`
- 启动: `flask --app server.py shell`
- 数据库操作: `db.create_all()`、`db.drop_all()`、`db.session.add_all()`、`db.session.commit()`

9.10 完整示例: Student Microblog

[p.181–193]

- 技术栈: Flask + SQLAlchemy + Jinja2 + Bootstrap + Flask-WTF
- 结构: `config.py` (配置) → `model.py` (ORM 模型: User、Post) → `studentForm.py` (表单: LoginForm) → `view.py` (视图函数) → `templates` (base.html、home.html、login.html) → `server.py` (启动)
- 功能: 用户登录验证、微博列表展示、闪存消息 (flashed messages)。

10 Trae AI 编程工具

10.1 Trae 工具简介

[p.197–199]

- **Trae** (The Real AI Engineer): 字节跳动 2025 年 1 月发布的全新 AI Coding 产品。
- 2025 年 11 月推出 SOLO 正式版; Skills 功能 2026 年 1 月正式上线。
- 三大能力:
 1. 需求理解: 支持自然语言输入, 解析模糊/复杂/矛盾需求

- 2. 工具调度：自动调用代码生成器、调试器、测试框架、Docker、Git 等
- 3. 自主执行：SOLO 模式下独立完成端到端开发任务
- 注册方式：手机 + 验证码 / 邮箱 / 社交媒体账号。

10.2 Trae 智能体（Agent）

[p.201–205]

智能体	能力描述
Chat	检查代码库或编写代码，不可生成代码文件；可优化输入内容后发送
Builder	可编辑文件、调用命令行终端、预览和联网搜索
Builder with MCP	在 Builder 基础上增加 MCP 协议调用功能
SOLO Coder	面向复杂项目，支持需求迭代到架构重构全流程；可添加自定义模型

- **MCP**（Model Context Protocol）：Anthropic 在 2024 年推出的开源协议，实现大语言模型与外部数据源和工具的安全双向连接。

10.3 Skill 创建与使用

[p.207–213]

- **Skill**：一套”专业能力说明书”，类似用户手册或操作指南（SOP）。
- **核心理念**：智能体 = 大模型（大脑）+ 工具（手脚）+ Skills（灵魂）。
- **使用场景**：高频重复操作、要求输出一致、固定工作流程、沉淀专业知识。
- **Skill 文件结构**：

```
my-skill/  
  SKILL.md          # 核心提示模板  
  scripts/          # 可执行脚本  
  references/        # 参考文档  
  assets/            # 模板和静态资源
```

- 可在 SOLO 模式下让 Trae 自动创建 Skill；支持修改和重命名。

10.4 Rule 创建与使用

[p.215–223]

- **Rule**：给 AI 功能生成结果添加规则和限制。优先级：用户输入 > 自定义 Agent Prompt > user_rules.md > project_rules.md。
- **主要场景**：
 - 提升效率：一次配置长期生效，减少沟通成本
 - 统一标准：团队规范结构化为规则
 - 保障质量：明确核心约束（架构、命名、风格等）

- **个人 Rule:** 存放于 C:\Users\<username>\.trae-cn\user_rules，所有项目生效，不被 git 追踪。
- **项目 Rule:** 存放于 .trae/rules，仅当前项目生效，被 git 追踪。
- **最佳实践:** 控制粒度、避免冲突、使用相对路径、优先引用强相关规则、新建规则后开启新对话。
[p.222-223]

11 总结：本章核心概念地图

板块	核心概念	关键页
系统工程	系统三要素（功能、结构、环境）、都江堰案例	4-18
软件特性	四大特性（研发/不磨损/定制/成本难估）、分类	23-26
软件危机	OS/360 案例、四方面危机表现、NATO 1968	28-34
可信性	可靠性 $R(t)$ 、可用性 $A(t)$ 、防危性风险公式、CIA 三要素	45-57
可信性工程	故障避免/移除/容错/预测、拜占庭容错	58-64
开发方法	结构化（功能）、OO（数据）、AO（关注点分离）、形式化（数学）	78-103
工具环境	三类工具、IDE、IPSE、ECMA 模型	105-117
AI 辅助开发	GAN/Transformer/Diffusion、千帆 API、Cline+DeepSeek	118-126
Flask 框架	WSGI、路由、Jinja2 模板、Bootstrap、WTF 表单、SQLAlchemy ORM	128-193
Trae 工具	四类 Agent、Skill 机制、Rule 机制	194-224